

OS Lab 08 Submission

Name: Yash Raj

Roll Number: 24k-0737

TASK 01:

```
#include <stdio.h>
#include <pthread.h>

#define MAX_THREADS 4
#define MAX_LINES 1000

char *lines[MAX_LINES];
int n = 0;
int count[MAX_THREADS];

typedef struct {
    int start;
    int end;
    int id;
} ThreadData;

void *count_lines(void *arg) {
    ThreadData *data = (ThreadData *)arg;
    int c = 0;
    for (int i = data->start; i < data->end; i++)
        c++;

    count[data->id] = c;
    return NULL;
}
```

```
int main() {
    pthread_t threads[MAX_THREADS];
    ThreadData tdata[MAX_THREADS];

    int chunk = n / MAX_THREADS;

    for (int i = 0; i < MAX_THREADS; i++) {
        tdata[i].start = i * chunk;
        tdata[i].end = (i == MAX_THREADS - 1) ? n : (i + 1) * chunk;
        tdata[i].id = i;
        pthread_create(&threads[i], NULL, count_lines, &tdata[i]);
    }

    for (int i = 0; i < MAX_THREADS; i++)
        pthread_join(threads[i], NULL);

    int total = 0;
    for (int i = 0; i < MAX_THREADS; i++)
        total += count[i];

    printf("Total Requests = %d\n", total);

    return 0;
}
```

```
PS D:\OS CODES\lab 8> gcc task1.c -o task1.exe
PS D:\OS CODES\lab 8> ./task1.exe
Total Requests = 8
PS D:\OS CODES\lab 8> █
```

TASK 02:

```
#include <stdio.h>
#include <pthread.h>

#define MAX_THREADS 4

int orders[] = {100, 200, 50, 300, 150, 80, 120, 60};
int n = 8;

int subtotal[MAX_THREADS];

typedef struct {
    int start;
    int end;
    int id;
} ThreadData;

void *sum_orders(void *arg) {
    ThreadData *data = (ThreadData *)arg;

    int sum = 0;
    for (int i = data->start; i < data->end; i++)
        sum += orders[i];

    subtotal[data->id] = sum;
    return NULL;
}

int main() {
    pthread_t threads[MAX_THREADS];
    ThreadData tdata[MAX_THREADS];

    int chunk = n / MAX_THREADS;

    for (int i = 0; i < MAX_THREADS; i++) {
        tdata[i].start = i * chunk;
        tdata[i].end = (i == MAX_THREADS - 1) ? n : (i + 1) * chunk;
        tdata[i].id = i;
        pthread_create(&threads[i], NULL, sum_orders, &tdata[i]);
    }

    for (int i = 0; i < MAX_THREADS; i++)
        pthread_join(threads[i], NULL);

    int total = 0;
    for (int i = 0; i < MAX_THREADS; i++)
        total += subtotal[i];

    printf("Total Revenue = %d\n", total);

    return 0;
}
```

- PS D:\OS CODES\lab 8> gcc task2.c -o task2.exe
- PS D:\OS CODES\lab 8> ./task2.exe
Total Revenue = 1060
- PS D:\OS CODES\lab 8> □

TASK 03:

```
#include <stdio.h>
#include <pthread.h>

#define N 8

int marks[N] = {55, 80, 45, 90, 60, 30, 75, 100};

float average;
int highest, lowest, passed;

void *calc_average(void *arg) {
    int sum = 0;
    for (int i = 0; i < N; i++)
        sum += marks[i];

    average = (float)sum / N;
    return NULL;
}

void *find_highest(void *arg) {
    highest = marks[0];
    for (int i = 1; i < N; i++)
        if (marks[i] > highest)
            highest = marks[i];

    return NULL;
}

void *find_lowest(void *arg) {
    lowest = marks[0];
    for (int i = 1; i < N; i++)
        if (marks[i] < lowest)
            lowest = marks[i];

    return NULL;
}

void *count_passed(void *arg) {
    passed = 0;
    for (int i = 0; i < N; i++)
        if (marks[i] >= 50)
            passed++;

    return NULL;
}
```

```
int main() {
    pthread_t t1, t2, t3, t4;

    pthread_create(&t1, NULL, calc_average, NULL);
    pthread_create(&t2, NULL, find_highest, NULL);
    pthread_create(&t3, NULL, find_lowest, NULL);
    pthread_create(&t4, NULL, count_passed, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_join(t3, NULL);
    pthread_join(t4, NULL);

    printf("Average Marks = %.2f\n", average);
    printf("Highest Marks = %d\n", highest);
    printf("Lowest Marks = %d\n", lowest);
    printf("Students Passed = %d\n", passed);

    return 0;
}
```

```
● PS D:\OS CODES\lab 8> gcc task3.c -o task3.exe
● PS D:\OS CODES\lab 8> ./task3.exe
Average Marks = 66.88
Highest Marks = 100
Lowest Marks = 30
Students Passed = 6
○ PS D:\OS CODES\lab 8> □
```

TASK 04:

```
#include <stdio.h>
#include <string.h>
#include <pthread.h>

#define MAX_THREADS 4
#define TEXT_SIZE 100

char text[] = "abc keyword xyz keywordabc keyword xyz keyword";
char keyword[] = "keyword";

int count[MAX_THREADS];

typedef struct {
    int start;
    int end;
    int id;
} ThreadData;

int match_at(int pos) {
    for (int i = 0; keyword[i] != '\0'; i++) {
        if (text[pos + i] != keyword[i])
            return 0;
    }
    return 1;
}

void *search(void *arg) {
    ThreadData *data = (ThreadData *)arg;

    int c = 0;
    int len = strlen(keyword);

    for (int i = data->start; i <= data->end - len; i++) {
        if (match_at(i))
            c++;
    }

    count[data->id] = c;
    return NULL;
}
```

```

int main() {
    pthread_t threads[MAX_THREADS];
    ThreadData tdata[MAX_THREADS];

    int n = strlen(text);
    int chunk = n / MAX_THREADS;
    int overlap = strlen(keyword) - 1;

    for (int i = 0; i < MAX_THREADS; i++) {
        tdata[i].start = i * chunk;

        if (i == MAX_THREADS - 1)
            tdata[i].end = n;
        else
            tdata[i].end = (i + 1) * chunk + overlap;

        tdata[i].id = i;

        pthread_create(&threads[i], NULL, search, &tdata[i]);
    }

    for (int i = 0; i < MAX_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    int total = 0;
    for (int i = 0; i < MAX_THREADS; i++) {
        total += count[i];
    }

    printf("Total occurrences of '%s' = %d\n", keyword, total);

    return 0;
}

```

```

● PS D:\OS CODES\lab 8> ./task4.exe
  Total occurrences of 'keyword' = 4
○ PS D:\OS CODES\lab 8> 

```

TASK 05:

```

#include <stdio.h>
#include <pthread.h>

#define MAX_THREADS 4

int numbers[] = {2, 3, 4, 5, 10, 13, 17, 20, 23, 29, 30, 31};
int n = 12;

int count[MAX_THREADS];

typedef struct {
    int start;
    int end;
    int id;
} ThreadData;

int is_prime(int num) {
    if (num < 2) return 0;
    for (int i = 2; i * i <= num; i++) {
        if (num % i == 0)
            return 0;
    }
    return 1;
}

void *count_primes(void *arg) {
    ThreadData *data = (ThreadData *)arg;

    int c = 0;
    for (int i = data->start; i < data->end; i++) {
        if (is_prime(numbers[i]))
            c++;
    }

    count[data->id] = c;
    return NULL;
}

```

```

int main() {
    pthread_t threads[MAX_THREADS];
    ThreadData tdata[MAX_THREADS];

    int chunk = n / MAX_THREADS;

    for (int i = 0; i < MAX_THREADS; i++) {
        tdata[i].start = i * chunk;
        tdata[i].end = (i == MAX_THREADS - 1) ? n : (i + 1) * chunk;
        tdata[i].id = i;

        pthread_create(&threads[i], NULL, count_primes, &tdata[i]);
    }

    for (int i = 0; i < MAX_THREADS; i++)
        pthread_join(threads[i], NULL);

    int total = 0;
    for (int i = 0; i < MAX_THREADS; i++)
        total += count[i];

    printf("Total Prime Numbers = %d\n", total);

    return 0;
}

```

```
● PS D:\OS CODES\lab 8> gcc task5.c -o task5.exe
● PS D:\OS CODES\lab 8> ./task5.exe
Total Prime Numbers = 8
○ PS D:\OS CODES\lab 8> █
```

TASK 06:

```
#include <stdio.h>
#include <pthread.h>

int balance = 1000;

void *deposit(void *arg) {
    int amount = *(int *)arg;

    balance = balance + amount;

    printf("Deposited %d\n", amount);
    return NULL;
}

void *withdraw(void *arg) {
    int amount = *(int *)arg;

    balance = balance - amount;

    printf("Withdrew %d\n", amount);
    return NULL;
}

int main() {
    pthread_t t1, t2, t3, t4;

    int d1 = 200, d2 = 300;
    int w1 = 150, w2 = 100;

    pthread_create(&t1, NULL, deposit, &d1);
    pthread_create(&t2, NULL, deposit, &d2);
    pthread_create(&t3, NULL, withdraw, &w1);
    pthread_create(&t4, NULL, withdraw, &w2);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_join(t3, NULL);
    pthread_join(t4, NULL);

    printf("Final Balance = %d\n", balance);

    return 0;
}
```



```

Total Prime Number = 0
PS D:\OS CODES\lab 8> gcc task6.c -o task6.exe
PS D:\OS CODES\lab 8> ./task6.exe
Deposited 200
Deposited 300
Withdraw 100
Withdraw 150
Final Balance = 1250
PS D:\OS CODES\lab 8> 

```

TASK 07:

```

#include <stdio.h>
#include <pthread.h>

#define FILES 3

char *filenames[FILES] = {"file1.txt", "file2.txt", "file3.txt"};
int counts[FILES];

typedef struct {
    int id;
} ThreadData;

void *count_words(void *arg) {
    ThreadData *data = (ThreadData *)arg;

    FILE *file = fopen(filenames[data->id], "r");
    if (file == NULL) {
        counts[data->id] = 0;
        return NULL;
    }

    int count = 0;
    char word[100];

    while (fscanf(file, "%s", word) != EOF) {
        count++;
    }

    counts[data->id] = count;

    fclose(file);
    return NULL;
}

```



```

int main() {
    pthread_t threads[FILES];
    ThreadData tdata[FILES];

    for (int i = 0; i < FILES; i++) {
        tdata[i].id = i;
        pthread_create(&threads[i], NULL, count_words, &tdata[i]);
    }

    for (int i = 0; i < FILES; i++) {
        pthread_join(threads[i], NULL);
    }

    int total = 0;

    for (int i = 0; i < FILES; i++) {
        printf("Words in %s = %d\n", filenames[i], counts[i]);
        total += counts[i];
    }

    printf("Total Word Count = %d\n", total);

    return 0;
}

```

```

Words in file1.txt = 8
Words in file2.txt = 4
Words in file3.txt = 5
Total Word Count = 17
PS D:\OS CODES\lab 8>

```

TASK 08:

```

#include <stdio.h>
#include <pthread.h>

#define MAX_THREADS 4

float ratings[] = {4.5, 3.0, 5.0, 4.0, 2.5, 4.8, 3.5, 5.0};
int n = 8;
float partial_sum[MAX_THREADS];

typedef struct {
    int start;
    int end;
    int id;
} ThreadData;

void *compute_sum(void *arg) {
    ThreadData *data = (ThreadData *)arg;

    float sum = 0.0;

    for (int i = data->start; i < data->end; i++)
        sum += ratings[i];

    partial_sum[data->id] = sum;
    return NULL;
}

```

```

int main() {
    pthread_t threads[MAX_THREADS];
    ThreadData tdata[MAX_THREADS];

    int chunk = n / MAX_THREADS;

    for (int i = 0; i < MAX_THREADS; i++) {
        tdata[i].start = i * chunk;
        tdata[i].end = (i == MAX_THREADS - 1) ? n : (i + 1) * chunk;
        tdata[i].id = i;

        pthread_create(&threads[i], NULL, compute_sum, &tdata[i]);
    }

    for (int i = 0; i < MAX_THREADS; i++)
        pthread_join(threads[i], NULL);

    float total_sum = 0.0;

    for (int i = 0; i < MAX_THREADS; i++)
        total_sum += partial_sum[i];

    float average = total_sum / n;

    printf("Overall Average Rating = %.2f\n", average);

    return 0;
}

```

- Total Word Count = 17
- PS D:\OS CODES\lab 8> gcc task8.c -o task8.exe
 - PS D:\OS CODES\lab 8> ./task8.exe
- Overall Average Rating = 4.04
- PS D:\OS CODES\lab 8> █

TASK 09:

```
#include <stdio.h>
#include <pthread.h>

#define N 6

int numbers[] = {2, 3, 4, 5, 6, 7};
int results[N];

int factorial(int n) {
    int fact = 1;
    for (int i = 1; i <= n; i++)
        fact *= i;
    return fact;
}

typedef struct {
    int start;
    int end;
    int id;
} ThreadData;

void *process_tasks(void *arg) {
    ThreadData *data = (ThreadData *)arg;

    for (int i = data->start; i < data->end; i++) {
        if (i % 3 == 0)
            results[i] = numbers[i] * numbers[i];
        else if (i % 3 == 1)
            results[i] = numbers[i] * numbers[i] * numbers[i];
        else
            results[i] = factorial(numbers[i]);
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;
    ThreadData d1 = {0, N/2, 0};
    ThreadData d2 = {N/2, N, 1};

    pthread_create(&t1, NULL, process_tasks, &d1);
    pthread_create(&t2, NULL, process_tasks, &d2);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Results:\n");
    for (int i = 0; i < N; i++)
        printf("Task %d (input %d) = %d\n", i, numbers[i], results[i]);

    return 0;
}
```

● PS D:\OS CODES\lab 8> gcc task9.c -o task9.exe

● PS D:\OS CODES\lab 8> ./task9.exe

Results:

Task 0 (input 2) = 4

Task 1 (input 3) = 27

Task 2 (input 4) = 24

Task 3 (input 5) = 25

Task 4 (input 6) = 216

Task 5 (input 7) = 5040

● PS D:\OS CODES\lab 8> □

TASK 10:

```
#include <stdio.h>
#include <pthread.h>

#define REGIONS 3
#define VOTES_PER_REGION 5

int votes[REGIONS][VOTES_PER_REGION] = {
    {1, 1, 0, 1, 1},
    {0, 1, 1, 0, 1},
    {1, 0, 1, 1, 0}
};

int region_count[REGIONS];

typedef struct {
    int region;
} ThreadData;

void *count_votes(void *arg) {
    ThreadData *data = (ThreadData *)arg;

    int sum = 0;

    for (int i = 0; i < VOTES_PER_REGION; i++) {
        sum += votes[data->region][i];
    }

    region_count[data->region] = sum;

    return NULL;
}
```

- PS D:\OS CODES\lab 8> gcc task10.c -o task10.exe
 - PS D:\OS CODES\lab 8> ./task10.exe
- Region-wise votes:
Region 1 = 4 votes
Region 2 = 3 votes
Region 3 = 3 votes
Total Votes = 10
- PS D:\OS CODES\lab 8>